Student Name: Om Kasvala.
Enrollment Number: 92410133005.
Division/Semester: 3EK2 / Semester 6.
Subject: HCD.
Course Assignment: TASK 5 - Teamwork/Individual to explore engineering tools , version control tool and continuous semester work.
Project Title: DevCare DeskBot: AI & IoT-Based Interactive System for Posture Awareness and Focus Improvement.

TASK 5 - Individual Project Exploration and Version Control

1. Individual Contribution and Role

For this project, I worked individually as the solo creator and developer. Because I did not have a team, my role covered every single part of the project from start to finish. I was responsible for designing the hardware setup (like wiring the ESP8266 microcontroller and the OLED screen) and writing all the software (the Python AI code for the laptop and the C++ code for the robot). To manage my work effectively, I broke the main project down into smaller, easy-to-finish tasks. For example, I first made sure the AI could detect posture on my laptop. Then, I built the physical robot. Finally, I focused on connecting them together using the AWS cloud. This step-by-step method kept me organized and focused.

I strongly applied Human-Centered Design (HCD) principles throughout my individual work. The entire core of this project was built around helping the user live a healthier and more productive life at their desk. I designed the system to watch out for the user's well-being by catching bad posture and sending a gentle alert. I also added features like hydration reminders and screen-break timers to prevent eye fatigue. To make sure the hardware was friendly to the user, I designed a simple "Do Not Disturb" touch sensor on the roof. This means the robot respects the user's time and adapts to their schedule, which is the most important rule of Human-Centered Design.

2. Version Control System Integration

To keep track of my project progress and manage all my code changes, I used **Git** as my Version Control System (VCS), along with **GitHub** to store my files safely online. Even when working alone, using a version control tool is extremely important. When you are writing complex code connecting AI, hardware, and the cloud, it is very easy to make a mistake that breaks the whole system. Git acted like a safety net for me.

Whenever I added a new feature that worked perfectly, I would "commit" (save) that version of the code. If I tried to add something new later and it caused the program to crash, I did not have to panic. I could simply use Git to roll back to my previous working version. I used a clear branching and commit strategy to keep my Python files and C++ files organized.

Here is an example of what my commit history looks like to show my step-by-step progress:

- *Commit 1:* "Initial setup for ESP8266 and basic OLED screen testing"
- *Commit 2:* "Added MediaPipe AI code for posture detection"
- *Commit 3:* "Set up AWS IoT Core certificates for cloud connection"
- *Commit 4:* "Updated screen layout code to fit the new 1.3-inch display"
- *Commit 5:* "Fixed the .exe packaging bug to hide the terminal window"'

  Github Link: https://github.com/ompatelok/DeskBot

Student Name: Om Kasvala.
Enrollment Number: 92410133005.
Division/Semester: 3EK2 / Semester 6.
Subject: HCD.
Course Assignment: TASK 5 - Teamwork/Individual to explore engineering tools , version control tool and continuous semester work.
Project Title: DevCare DeskBot: AI & IoT-Based Interactive System for Posture Awareness and Focus Improvement.

3. Reflection & Challenges

Managing this entire project individually was a huge learning experience for me. It taught me how to be my own project manager, how to stick to a timeline, and how to solve problems without waiting for someone else to help. It improved my confidence in handling both hardware and software at the same time.

However, working alone also brought some tough challenges. One major challenge was getting the laptop AI to talk to the physical robot through the AWS cloud. Sometimes, the connection would fail because the code could not find the right security certificates. Another big challenge was turning my final Python script into a simple `.exe` application without the program crashing due to missing files. I solved these problems by taking things slowly and testing one small piece at a time. I used the computer terminal to read the exact red error messages so I knew exactly what file was missing. By isolating the problems and fixing them step-by-step, I was able to successfully finish the project on my own.